Mechanism-Aware AI Assurance: An Operational-Premise Taxonomy for Artificial Intelligence

Wesley R. Elsberry

July 2, 2026

Abstract

AI governance frameworks increasingly organize accountability through inventories, risk tiers, documentation, monitoring, human oversight, and post-deployment review. These controls are necessary, but they can remain mechanistically incomplete: a system may satisfy process expectations while still failing to identify the known failure modes of the computation that actually produces its behavior. We formalize the *Operational-Premise Taxonomy* (OPT), a compact mechanism-level classification for artificial intelligence systems. OPT classifies systems by operative computational premise rather than by product label or training-signal shorthand, using seven roots: **Lrn** for parametric learning, **Evo** for evolutionary adaptation, **Sym** for symbolic reasoning, **Prb** for probabilistic inference, **Sch** for search and planning, **Ctl** for control and feedback, and **Swm** for swarm or collective behavior. We provide core mathematical operators, link the roots to canonical biological mechanisms, and define a compositional grammar for hybrids. We further introduce OPT–Intent as a design-time record of intended mechanisms, assumptions, constraints, risks, and assurance evidence, and OPT–Code as an implementation-time record of mechanisms actually present. The resulting comparison exposes mechanism drift and helps connect governance evidence to mechanism-specific failure modes. OPT is therefore proposed as a governance-compatible assurance layer: not a replacement for NIST, ISO, OECD, or regulatory frameworks, but a mechanism vocabulary that helps determine which tests, monitors, and residual-risk decisions are technically relevant.

1 Introduction

Current AI governance practice rightly emphasizes accountability, intended use, risk management, documentation, monitoring, and human oversight. Yet those processes can be incomplete if they do not identify the operative mechanism that creates system behavior. A risk register that labels a system only as "AI," "machine learning," or "automated decision support" may say little about the failure modes that matter for review. A learned model, a symbolic rule engine, a probabilistic inference system, a search optimizer, a feedback controller, an evolutionary search process, and a swarm system require different assurance evidence.

Regulatory and industry texts nevertheless often compress AI into three categories of *learning signals*: supervised, unsupervised, and reinforcement learning [EUA, 2024, Tabassi et al., 2023]. These categories emerged from neural/connectionist practice, not from the full breadth of artificial intelligence [Russell and Norvig, 2020]. They are also poorly suited to mechanism-level

assurance. They do not describe whether a deployed system reasons over rules, updates a belief state, searches an objective landscape, controls an actuator, evolves a population, or coordinates through local collective behavior.

We propose an alternative taxonomic axis: the *operational premise*—the primary computational mechanism a system instantiates to improve, adapt, infer, optimize, decide, or act. The resulting *Operational-Premise Taxonomy* (OPT) provides a transparent and consistent framework for compactly describing AI systems, including hybrids and pipelines. OPT retains biological analogs (learning vs. adaptation) while accommodating symbolic, probabilistic, search, control, and swarm paradigms. Its purpose is not merely to name systems more neatly. Its purpose is to make mechanism-specific assurance questions visible at design time, implementation time, and review time.

The contributions of this paper are:

1. a seven-root mechanism taxonomy spanning **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, and **Swm**;
2. a compact hybrid grammar for systems that combine mechanisms;
3. mathematical and biological grounding for each root;
4. an assurance-oriented account of mechanism-specific and joint mechanism risk;
5. OPT–Intent and OPT–Code artifacts for design-time declaration, implementation-time classification, and drift review.

2 The Assurance Gap: Governance Without Mechanism

AI governance frameworks have become the dominant way to organize accountability for artificial intelligence systems. Frameworks such as the NIST AI Risk Management Framework, ISO/IEC AI standards, OECD classification work, and the EU AI Act establish essential process controls: ownership, intended use, risk management, documentation, monitoring, human oversight, and post-deployment review. These controls are necessary. They are not sufficient when they remain mechanism-agnostic.

The missing layer is mechanism-level assurance. A system can be documented, assigned an owner, placed in a risk tier, and subjected to lifecycle review while still failing to ask whether the operative computational mechanism has known failure modes that matter in the deployment context. Mechanism is not a minor implementation detail. A learned ranking model, a probabilistic nowcasting system, a search optimizer, a rule engine, and a feedback controller create different assurance questions even when each is labeled "AI" or "automated decision support."

This omission is not merely theoretical. High-cost software-intensive failures often involve a mismatch between broad process confidence and unchecked operational assumptions. Ariane 5 Flight 501 exposed unsafe reuse and numeric range assumptions in flight software [Ariane 501 Inquiry Board, 1996]. Mars Climate Orbiter exposed interface and unit assumptions across a software-intensive system boundary [Mars Climate Orbiter Mishap Investigation Board, 1999]. Knight Capital showed how deployment-state and stale-code assumptions can create large automated trading losses [U.S. Securities and Exchange Commission, 2013]. The Boeing 737 MAX MCAS accidents highlighted control, sensor, authority, and human-response assumptions in a feedback system [National Transportation Safety Board, 2019]. The 2003 Northeast blackout involved observability and control-room alarm failures in a coupled operational

system [U.S.-Canada Power System Outage Task Force, 2004]. The 2010 flash crash showed automated execution and market feedback interacting through search/control-like mechanisms [U.S. Commodity Futures Trading Commission and U.S. Securities and Exchange Commission, 2010]. Google Flu Trends illustrates how learned or probabilistic proxy inference can degrade under drift, social feedback, and weak recalibration against ground truth [Lazer et al., 2014].

These cases do not imply that OPT would have prevented each incident. The more defensible claim is narrower and more useful: a mechanism-aware review makes specific classes of technical assumptions visible earlier. Once a system is classified as **Lrn**, **Prb**, **Sch**, **Ctl**, **Sym**, **Evo**, **Swm**, or a hybrid, assurance can ask mechanism-specific questions:

- For **Lrn**, what evidence addresses biased labels, leakage, distribution shift, reward or loss misspecification, and calibration?
- For **Prb**, are priors, likelihood assumptions, uncertainty estimates, and approximations valid in the operational setting?
- For **Sch**, what objective is optimized, which constraints are hard, and what prevents pathological optima?
- For **Ctl**, what are the stability margins, sensor assumptions, actuator authority, degraded modes, and human override conditions?
- For **Sym**, are rules traceable, consistent, complete enough for use, and robust to exceptions?
- For **Evo**, does the fitness function represent the desired outcome, and what prevents unsafe candidate exploration?
- For **Swm**, what local rules generate the global behavior, and what bounds emergent instability?

OPT therefore addresses an assurance gap rather than replacing governance. Governance frameworks specify who is accountable and what kinds of evidence must exist. OPT helps identify which evidence is relevant by naming the operative mechanism and the joint risk profile of hybrids. OPT–Intent extends that benefit to design time by recording intended mechanisms, assumptions, constraints, risks, and release-blocking evidence before implementation hardens around implicit choices. OPT–Code records the mechanisms actually present in the implemented system. Comparing the two exposes mechanism drift: for example, a nominally symbolic system that quietly gains learned ranking, a decision-support system that becomes de facto control, or a forecasting model that becomes part of an optimization loop.

This framing also explains why training-signal labels are inadequate. The problem is not only that "supervised," "unsupervised," and "reinforcement" omit many AI traditions. The deeper problem is that such labels are too coarse to drive assurance. A governance process needs a compact way to ask what kind of computation produces behavior, how mechanisms combine, what failure modes follow from that composition, and which tests or monitors make residual risk explicit. OPT is proposed as that mechanism layer.

3 Operational-Premise Taxonomy (OPT)

Because OPT introduces several new labels, we present those here before tackling background and related work topics. (See also Appendix A.)

OPT classes are defined by dominant mechanism; hybrids are explicit compositions:

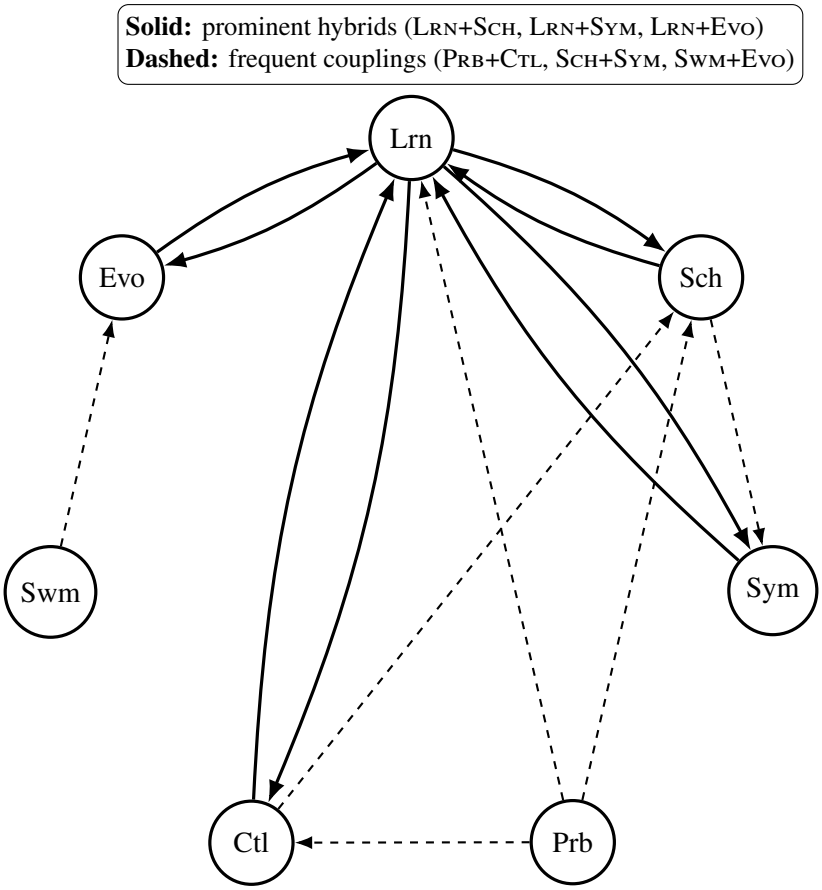

Figure 1: OPT landscape using short names only: **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**.

- **Learnon (Lrn)** — Parametric learning within an individual (gradient/likelihood/return updates).
- **Evolon (Evo)** — Population adaptation via variation, selection, inheritance.
- **Symbion (Sym)** — Symbolic/logic inference over discrete structures (KB, clauses, proofs).
- **Probion (Prb)** — Probabilistic modeling and approximate inference (posteriors, ELBO).
- **Scholon (Sch)** — Deliberative search and planning (heuristics, DP, graph search).
- **Controlon (Ctl)** — Feedback control and state estimation in dynamical systems.
- **Swarmon (Swm)** — Collective/swarm coordination with local rules and emergence.

Hybrid notation. We use A+B for co-operative mechanisms, A/B for hierarchical nesting (outer/inner), A{B,C} for parallel ensembles, and [A→B] for pipelines (Appendix B).

4 Background and Prior Work

Classic textbooks and surveys treat symbolic reasoning, planning/search, probabilistic models, learning, evolutionary methods, and control/estimation as co-equal pillars [Russell and Norvig, 2020, Fogel et al., 2016, Alcala-Fdez and Alonso, 2016, Sutton and Barto, 2018]. No-Free-Lunch (NFL) theorems for search/optimization motivate pluralism: no single mechanism

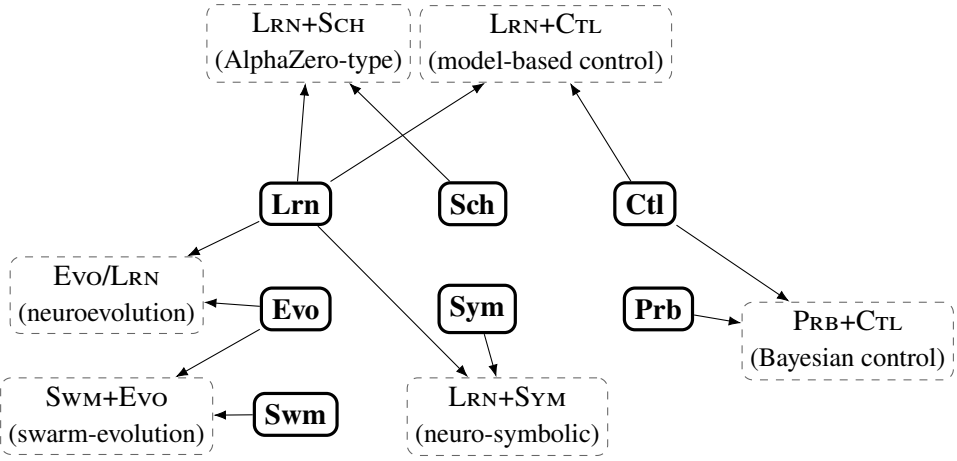

Figure 2: Hybrid "ancestry" diagram: short-name roots (solid) and exemplar hybrids (dashed).

dominates across all problems [Wolpert and Macready, 1997]. Biological literatures mirror these mechanisms: synaptic plasticity and Hebbian/Oja learning [Hebb, 1949, Oja, 1982], population genetics and replicator dynamics [Price, 1970, Taylor and Jonker, 1978], Bayesian cognition [Knill and Pouget, 2004], and optimal feedback control in motor behavior [Todorov and Jordan, 2002, Kalman, 1960, Pontryagin et al., 1962].

5 Related Work: Existing Taxonomies and Frameworks

Standards bodies and policy groups have invested heavily in AI definitions, lifecycle models, and governance instruments. However, none provides a compact, mechanism-centric taxonomy spanning **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, and **Swm**, nor an explicit grammar for hybrids.

Standards and terminology. ISO/IEC 22989 standardizes terms and core concepts for AI across stakeholders, serving as a definitional foundation rather than a technique taxonomy. ISO/IEC 23053 offers a functional block view for *machine-learning-based* AI systems (data, training, inference, monitoring), which is valuable architecturally but limited to ML and therefore excludes non-ML pillars such as symbolic reasoning, control/estimation, and swarm/evolutionary computation [ISO, 2022a,b].

Risk and management frameworks. NIST's AI Risk Management Framework (AI RMF 1.0) provides an implementation-agnostic process for managing AI risks (govern, map, measure, manage). Its companion *AI Use Taxonomy* classifies human–AI task interactions and use patterns. Both are intentionally technique-agnostic: they can apply to any implementation class, but do not sort systems by operative mechanism [Tabassi et al., 2023, Theofanos et al., 2024].

Policy classification tools. The OECD Framework for the Classification of AI Systems organizes systems along multi-dimensional policy axes (People & Planet, Economic Context, Data & Input, AI Model, Task & Output). This is a powerful policy characterization instrument, yet it remains descriptive and multi-axis rather than a compact mechanism taxonomy with hybrid syntax [OEC, 2022].

Regulatory regimes. The EU Artificial Intelligence Act introduces risk-based classes (e.g., prohibited, high-risk, limited, minimal) and obligations, largely orthogonal to implementation specifics. Technique details matter for *compliance evidence*, but the Act does not define a canonical implementation taxonomy [EUA, 2024].

Academic precedents and surveys. The textbook tradition organizes AI by substantive pillars—search/planning, knowledge/logic, probabilistic reasoning, learning, and agents—closely aligning with the mechanism families in this paper but without proposing a stable naming code or formal hybrid grammar [Russell and Norvig, 2020]. Reinforcement learning texts formalize optimization and value iteration for **Lrn/Sch** couplings [Sutton and Barto, 2018]. Classical theory anchors **Prb** (Knill and Pouget, 2004), **Ctl** (Kalman, 1960, Pontryagin et al., 1962, Todorov and Jordan, 2002), and foundational dynamics for **Evo** (Price, 1970, Taylor and Jonker, 1978). Learning rules for **Lrn** include Hebbian and Oja's formulations [Hebb, 1949, Oja, 1982], while resolution proofs formalize **Sym** [Robinson, 1965]. No-Free-Lunch results motivate preserving multiple mechanisms rather than collapsing them into a single "optimization" bucket [Wolpert and Macready, 1997].

Gap and contribution. Taken together, these works motivate *two layers*: (i) policy/lifecycle/risk instruments that are technique-agnostic and (ii) a compact, biologically grounded *implementation taxonomy* with explicit hybrid composition. OPT fills the second layer with seven frozen roots and a grammar for hybrids, designed to interface cleanly with the first layer.

Comparative landscape. Table 1 situates OPT alongside the best-known standards, policy instruments, and textbook structures. Each of these prior frameworks serves an important function—shared vocabulary (ISO/IEC 22989), ML-system decomposition (ISO/IEC 23053), risk management (NIST AI RMF), usage contexts (NIST AI 200-1), multidimensional policy characterization (OECD), or regulatory stratification (EU AI Act). However, they remain either technique-agnostic or focused solely on machine learning. OPT complements them by supplying the missing layer: a stable, biologically grounded *implementation taxonomy* that captures mechanism families across paradigms and defines a formal grammar for hybrid systems.

6 Comparative Analysis, Completeness, and Objections

6.1 Biological–Artificial Correspondences

Each OPT class aligns with a biological mechanism (plasticity, natural selection, structured reasoning, Bayesian cognition, deliberative planning, optimal feedback control, and distributed coordination). Shared operators in Sec. 14 support cross-domain guarantees.

6.2 Coverage, Hybrids, and Orthogonal Descriptors

Hybrids are explicit (e.g., Lrn+Sch AlphaZero, Lrn+Sym neuro-symbolic, Evo/Lrn neuroevolution). Orthogonal axes capture representation, locus of change, objective, data regime, timescale, and human participation.

[t]

Table 1: Comparison of OPT with existing standards, policy frameworks, and textbook pillars.

Framework / Source	**Primary Scope**	**Unit of Classification**	**Technique Coverage**	**Hybrid Handling / Intended Use**
OPT (this work)	Implementation taxonomy	*Operative mechanism* (**Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**) with composition grammar	Cross-paradigm (learning, symbolic, probabilistic, search, control, swarm, evolutionary)	Explicit hybrids via +, /, {}, [→]; designed to interface with risk/process frameworks
ISO/IEC 22989:2022 [ISO, 2022a]	Concepts & terminology	Vocabulary / definitions	Technique-agnostic	No hybrid grammar; supports common language across stakeholders
ISO/IEC 23053:2022 [ISO, 2022b]	ML system architecture	Functional blocks (data, training, inference, monitoring)	ML-centric; excludes non-ML pillars (e.g., **Sym**, **Ctl**, **Swm**)	No explicit hybrid mechanism model; system design/process lens
NIST AI RMF 1.0 [Tabassi et al., 2023]	Risk management	Risk functions (Govern, Map, Measure, Manage)	Technique-agnostic	No mechanism taxonomy; governance and assurance guidance
NIST AI 200-1 [Theofanos et al., 2024]	Use taxonomy	Human–AI task activities	Technique-agnostic	No hybrids; categorizes use contexts for evaluation
OECD AI Classification [OEC, 2022]	Policy characterization	Multi-axis profile (context, data, model, task)	Broad; includes an "AI model" axis but not a formal mechanism taxonomy	No hybrid grammar; policy comparison and statistics
EU AI Act [EUA, 2024]	Regulation (risk-based)	Risk class (prohibited/high/limited/minimal)	Technique-agnostic	Hybrids irrelevant; compliance and obligations
AIMA (Russell & Norvig) [Russell and Norvig, 2020]	Textbook organization	Pillars (search/planning, logic, probabilistic reasoning, learning, agents)	Broad coverage; closest to mechanism families	No standard naming or hybrid code; educational structure

Notes. OPT supplies a compact, biologically grounded *implementation* taxonomy with a formal hybrid composition code. Standards and policy frameworks remain essential and complementary for vocabulary, lifecycle, risk, management, and regulatory obligations, but they are technique-agnostic or ML-specific and do not provide a mechanism-level naming scheme.

6.3 Objections and Responses

Reduction to optimization. Mechanisms imply distinct guarantees/hazards (data leakage vs. fitness misspecification vs. rule brittleness). NFL cautions against collapsing mechanisms. **Hybrid blurring.** OPT treats compositions as first-class; the notation discloses "what changes where, on what objective, and on what timescale." **Regulatory simplicity.** Seven bins appear minimal for coverage; the short names keep disclosures compact and meaningful.

7 Comparison of OPT with Other AI Classification Frameworks

Feature	**Supervised / Unsupervised**	**Symbolic / Sub-symbolic**	**OECD / Policy Frameworks**	**Model Cards / ADR**	**OPT**
Mechanism-Level Classification	No	Partial	No	No	Yes
Supports Hybrid Systems Explicitly	Limited	Limited	No	No	Yes
Biological Correspondence	Partial	Limited	No	No	Yes
Covers Evolutionary Methods	No	Partial	No	No	Yes
Covers Control Systems	No	No	No	No	Yes
Covers Swarm Methods	No	No	No	No	Yes
Formal Grammar Defined	No	No	No	No	Yes
Supports Governance Mapping	Indirect	No	Yes	Yes	Yes
Detects Mechanism Drift	No	No	No	No	Yes
Supports Agentic AI Reasoning	No	No	Limited	No	Yes
Enables Mechanism-Guided Remediation	No	No	No	No	Yes

Table 2: Representative paradigms mapped to OPT.

Type / Implementation	Examples	OPT (short)
NN/Transformer (GD)	CNN, LSTM, attention	**Lrn**
Reinforcement learning	DQN, PG, AC	**Lrn** (+**Sch**, +**Ctl**)
Evolutionary algorithms	GA, GP, CMA-ES	**Evo**
Swarm intelligence	ACO, PSO	**Swm** (+**Evo**)
Expert systems	Prolog, Mycin, XCON	**Sym**
Probabilistic models	BN, HMM, factor graphs	**Prb**
Search & planning	A*, MCTS, STRIPS	**Sch**
Control & estimation	PID, LQR, KF/MPC	**Ctl**

8 Examples and Mapping

9 Orthogonal Axes and Risk Perspectives

Secondary axes (orthogonal descriptors).

- **Representation:** parametric vectors, symbols/logic, graphs, programs, trajectories, policies.
- **Locus of Change:** parameters, structure/architecture, population composition, belief state, policy.
- **Objective Type:** prediction, optimization, inference, control, search cost, constraint satisfaction.
- **Timescale:** online vs. offline; within-run vs. across-generations.
- **Data Regime:** none/synthetic, labeled, unlabeled, interactive reward.
- **Human Participation:** expert-authored knowledge vs. learned vs. co-created.

9.1 Artificial Immune Systems (AIS) in OPT

It is useful to show how OPT-code specifications can be derived for examples of a technique that is a hybrid.

Artificial Immune Systems (AIS) instantiate computation via biomimetic mechanisms drawn from adaptive immunity. Their operative core combines (i) population-level *variation and selection* (somatic hypermutation, clonal expansion, memory) and (ii) distributed, locally interacting agents (cells, idiotypic networks), often with (iii) probabilistic fusion of uncertain signals. In OPT, this places AIS primarily in **Evo** and **Swm**, with frequent couplings to **Prb** and occasional **Sch/Ctl** layers depending on task and implementation.

Table 3: Orthogonal descriptive axes and governance risks (abridged).

OPT	Primary Risks	Assurance Focus
Lrn	Data leakage, reward hacking	Data governance, OOD tests, calibration
Evo	Fitness misspecification	Proxy validation, replicates, constraints
Sym	Rule brittleness, KB inconsistency	Provenance, formal verification
Prb	Miscalibration, inference bias	Posterior predictive checks
Sch	Heuristic inadmissibility	Optimality proofs, heuristic diagnostics
Ctl	Instability, unmodeled dynamics	Stability margins, robustness
Swm	Emergent instability	Swarm invariants, safety envelopes

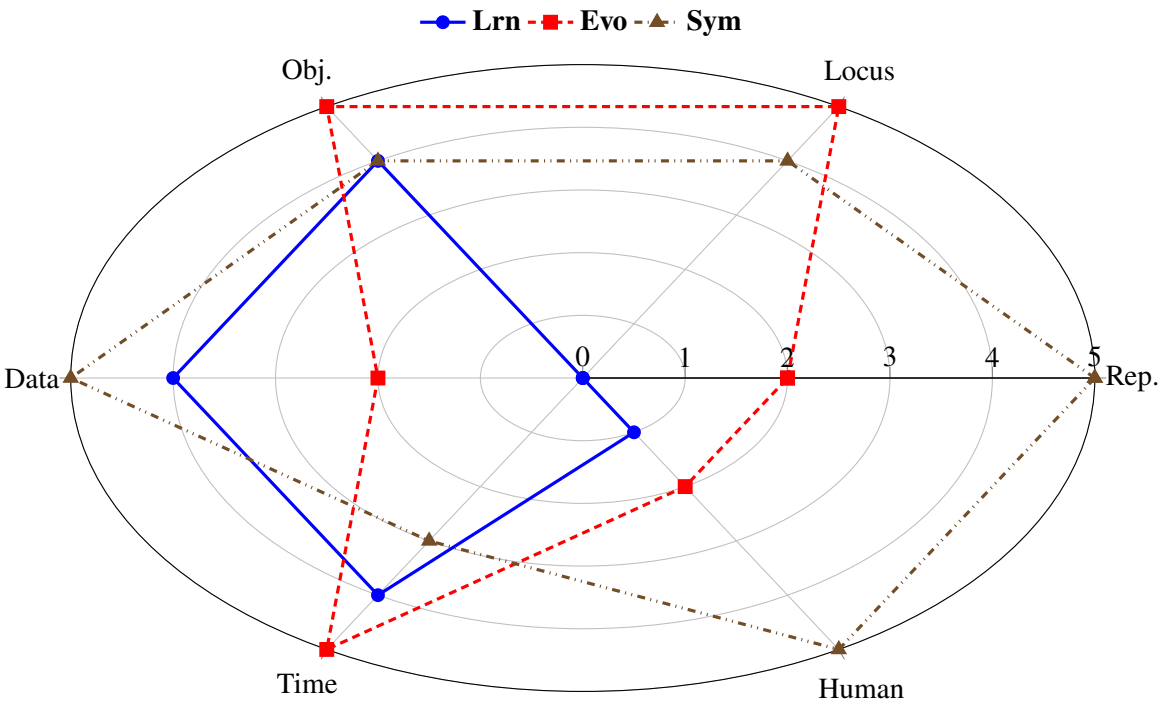

Figure 3: Orthogonal axes (0–5) for **Lrn**, **Evo**, **Sym**.

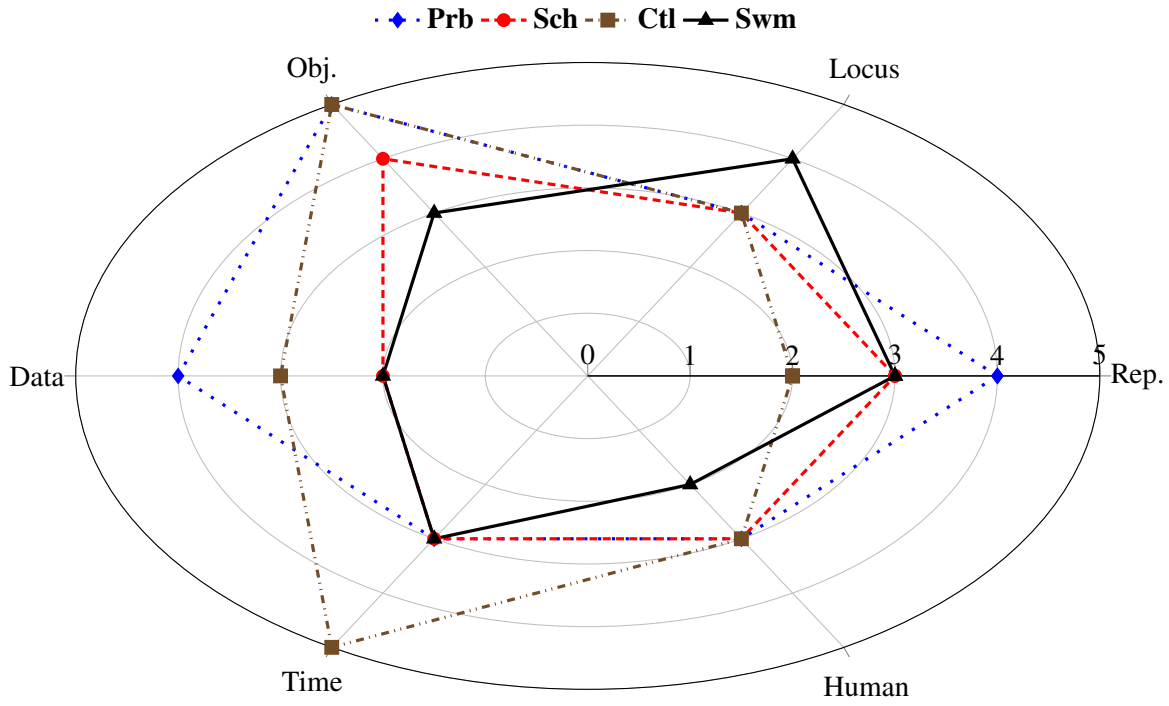

Figure 4: Orthogonal axes (0–5) for **Prb**, **Sch**, **Ctl**, **Swm**.

Canonical families and OPT placement.

- **Clonal selection & affinity maturation (CLONALG, aiNet).** Population of detectors/antibodies $\{a_i\}$ undergo clone–mutate–select cycles driven by affinity to antigens x. OPT: **Evo+Swm** (often +**Prb**).
  Affinity (bitstrings; Hamming distance d_H): $\mathrm{aff}(x,a) = 1 - \frac{d_H(x,a)}{|x|}$. Clone count $n_i \propto \mathrm{aff}(x,a_i)$; hypermutation rate $\mu_i = f(\mathrm{aff})$ (typically inversely proportional).

- **Negative Selection Algorithms (NSA).** Generate detectors that avoid "self" set $\mathcal{S}$ and cover $\mathcal{X} \setminus \mathcal{S}$. OPT: **Evo/Sch** (+**Prb** for thresholded matching).
  Objective: choose D s.t. $\forall d \in D : d \notin \mathcal{S}$ and coverage $\Pr[\mathrm{match}(x,d) \mid x \notin \mathcal{S}] \geq \tau$.

- **Immune network models (idiotypic).** Interacting clones stimulate/suppress each other; dynamics produce memory and regulation. OPT: **Swm+Evo** (sometimes +**Ctl**).
  Skeleton dynamics: $\dot{a}_i = \sum_j s_{ij} a_j - \sum_j \sigma_{ij} a_i a_j - \delta a_i$ with stimulation s_{ij}, suppression σ_{ij}, decay δ.

- **Dendritic Cell Algorithm (DCA) / Danger Theory.** Cells fuse PAMP/danger/safe signals to decide anomaly labeling; aggregation over a population provides robust detection. OPT: **Swm+Prb** (optionally +**Evo** if online adaptation is added).

OPT-Code exemplars.

```
CLONALG: OPT=Evo+Swm; Rep=bitstring; Obj=affinity; Data=labels|unlabeled;
Time=gens; Human=low
aiNet:  OPT=Evo+Swm; Rep=realvector; Obj=affinity+diversity; Time=gens
NSA (anomaly):  OPT=Evo/Sch+Prb; Rep=bitstring; Obj=coverage; Data=self/nonself;
Time=gens
```

```
DCA: OPT=Swm+Prb; Rep=signals; Obj=anomaly-score; Time=online
Idiotypic control:  OPT=Swm+Ctl; Rep=rules; Obj=stability+coverage; Time=online
```

Where biology and OPT coincide. Somatic hypermutation+ selection → **Evo**; massive agent concurrency and local rules → **Swm**; uncertainty fusion (signal weighting, thresholds) → **Prb**; homeostatic regulation → **Ctl**; detector-set coverage and complement generation → **Sch**.

Assurance considerations. Key failure modes are coverage gaps (missed anomalies), detector drift, and instability in network dynamics. Assurance suggests (i) held-out self/non-self tests, (ii) diversity and coverage metrics, (iii) stability analysis of interaction graphs, and (iv) calibration of anomaly thresholds (if **Prb**). These layer cleanly with risk/management frameworks (NIST RMF, ISO 23053) while OPT communicates mechanism.

10 Design and Governance with OPT–Intent and OPT–Code

A central motivation for the Operational Premise Taxonomy (OPT) is to support not only the analysis of existing AI systems, but also the design and governance of systems throughout their lifecycle. Most established AI documentation frameworks focus on models that already exist—for example, Model Cards, AI Service Cards, or post-hoc documentation embedded in software-engineering artefacts. In contrast, OPT provides an explicit mechanism-level vocabulary that can be applied *before*, *during*, and *after* implementation.

To this end, we distinguish two complementary artefacts: *OPT–Intent*, a design-time declaration of planned mechanisms, goals, constraints, and risks; and *OPT–Code*, a run-time classification of the system as implemented. Together, these artefacts form a governance substrate that is lightweight, expressive, and compatible with software-architecture practices and AI governance frameworks. Their assurance role is to tie general governance obligations to mechanism-specific evidence: the tests, monitors, constraints, and residual-risk decisions that are relevant because of how the system actually computes.

10.1 OPT–Intent as a Design-Time Mechanism Declaration

OPT–Intent expresses the *intended* operative mechanisms (**Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**), the domain-level goal, key constraints, anticipated risks, and the deployment context. The notation supports early-stage architectural reasoning:

```
INTENT-OPT = Sch/Evo
INTENT-GOAL = robust-production-schedule-under-dynamic-constraints
INTENT-CONSTRAINTS = real-time, explainable, limited-human-oversight
INTENT-RISKS = local-minima, premature-convergence, objective-misspecification
INTENT-EVIDENCE = constraint-tests, stress-cases, monitor-thresholds
INTENT-CONTEXT = manufacturing-decision-support
```

This declaration resembles a focused architectural decision record (ADR), but is grounded in OPT's mechanism vocabulary. Whereas classical goal-oriented requirements engineering (GORE) frameworks such as KAOS, i, or Tropos provide high-level goal models, OPT–Intent provides a mechanism-centered annotation that connects those goals to families of computational approaches.

10.2 OPT–Code as an Implementation-Time Mechanism Description

Once a system is implemented, its operative mechanisms can be classified via OPT–Code:

```
OPT=Evo/Sch/Sym; Rep=permutations+rules; Obj=production-cost;
Data=inventory+constraints; Time=generations+online-adjust; Human=medium
```

OPT–Code reflects the *actual* mechanisms as they appear in the final architecture and implementation. Comparing OPT–Intent with OPT–Code provides a principled way to detect architectural drift, unplanned mechanism additions, and deviations from original constraints.

10.3 Alignment Analysis: Intent vs. Implementation

The relationship between OPT–Intent and OPT–Code supports alignment analysis across the AI system lifecycle:

- **Mechanism alignment:** Whether the realized mechanisms match the intended roots, or whether additions (e.g., **Sym** for explainability) or substitutions introduce new behavior.
- **Objective alignment:** Whether the objective in OPT–Code (Obj) is consistent with the purpose in INTENT-GOAL.
- **Constraint alignment:** Whether the implementation respects INTENT-CONSTRAINTS (e.g., real-time, explainability, human oversight).
- **Risk evolution:** Whether realized mechanisms introduce additional risks relative to INTENT-RISKS (e.g., adding learned components introduces data-dependence).

This analysis can be automated with an "OPT–Intent Alignment Evaluator" in LLM-based workflows, producing an alignment verdict and score.

10.4 Integration with Existing Governance Artefacts

OPT aligns with and supplements existing governance frameworks:

AI Documentation (Model Cards, AI Service Cards). Model Cards and similar frameworks capture purpose, data provenance, limitations, and performance characteristics of trained models. These artefacts begin at model completion. OPT–Intent supplements them with a *design origin*, and OPT–Code provides a *mechanism-centric summary* useful for governance, reproducibility, and safety assessments.

Architecture Decision Records (ADRs). ADRs record the rationale for major architectural decisions. OPT–Intent functions as a structured, mechanism-focused ADR, intended to be referenced in downstream ADRs describing implementation choices and trade-offs.

Safety, Risk, and Impact Assessments. Regulatory frameworks such as the OECD AI classification or NIST AI Risk Management Framework classify AI systems according to use, risk, and context. OPT complements these by classifying operative mechanisms. Mechanism-level classification is critical because risk profiles are often mechanism-dependent: population-based adaptation (**Evo**), closed-loop control (**Ctl**), probabilistic inference (**Prb**), search/optimization (**Sch**), and learned representation (**Lrn**) each generate distinct failure modes and therefore require distinct assurance evidence.

GORE and Requirements Engineering. OPT–Intent is compatible with KAOS, i, and Tropos goal structures, providing a compact mapping from stakeholder goals to operative mechanisms. Instead of treating "use AI" as a monolithic design choice, OPT forces the mechanism to be named explicitly.

10.5 Lifecycle Governance with OPT

An AI system moves through phases of design, implementation, deployment, revision, and decommissioning. OPT supports governance at each phase:

1. **Design:** Authors specify OPT–Intent and identify mechanistic justifications and constraints.
2. **Implementation:** OPT–Code is generated and compared with Intent for architectural drift.
3. **Evaluation:** OPT classifiers, evaluators, and adjudicators check mechanism correctness, formatting, and risk implications.
4. **Deployment:** OPT–Code informs safety monitoring, audit logs, and mechanism-specific risk controls (e.g., for **Ctl** or **Evo** systems).
5. **Revision and re-training:** OPT alignment is reassessed when system behavior changes or new mechanisms are introduced.
6. **Documentation & reporting:** OPT–Intent and OPT–Code form part of a long-term audit trail, linking design rationale to implemented system behavior.

10.6 AI Design Assistants and Automated Governance

OPT also provides a structured interface for LLM-based design assistants. Given a functional goal or stakeholder requirement, an OPT-aware model can produce candidate OPT–Intent declarations and propose mechanism families suitable for achieving the goal. Downstream evaluation and adjudication prompts make it possible to manage and audit these proposals automatically.

Such workflows enable a novel form of governance: mechanism-level traceability. Instead of asking only whether a system is "fair," "safe," or "performant," practitioners can ask whether its mechanisms match the intended design, whether mechanism additions add new risks, and whether the alignment between purpose and implementation is conserved over time. OPT thus becomes a bridge between requirements engineering, architectural practice, risk governance, and the technical analysis of AI systems.

11 OPT and Agentic AI Workflows

Recent advances in agentic artificial intelligence emphasize systems that plan, act, evaluate, and repair their own behavior through iterative interaction with tools, environments, and internal models. Such systems typically decompose goals, invoke tools, assess outcomes, and revise plans in closed loops. While these architectures have proven powerful, they frequently lack an explicit representation of the *operative mechanisms* through which actions are taken and errors

arise. This omission complicates reasoning about failure modes, governance constraints, and design trade-offs.

The Operational Premise Taxonomy (OPT) provides a mechanism-level abstraction layer that can be integrated into agentic workflows to address these gaps. Rather than prescribing a particular agent architecture, OPT supplies a shared vocabulary and analytical framework that agentic systems can use to reason about how tasks are performed, how errors should be interpreted, and how repairs should be constrained.

11.1 Mechanism Awareness in Agentic Systems

Agentic workflows are often described in terms of high-level functional stages (planning, execution, critique, repair), but these stages are agnostic to the computational mechanisms employed. In practice, however, the behavior and risk profile of an agentic system depend critically on whether its actions rely on parametric learning (**Lrn**), symbolic reasoning (**Sym**), search (**Sch**), probabilistic inference (**Prb**), control (**Ctl**), evolutionary adaptation (**Evo**), or swarm dynamics (**Swm**), or some hybrid combination thereof.

OPT introduces explicit mechanism awareness into agentic reasoning. An OPT-aware agent can classify its own components, tools, or subplans in terms of OPT roots, enabling it to reason not merely about *what* is being done, but about *how* it is being done. This distinction becomes especially important in hybrid agentic systems that combine learning-based components with search, symbolic constraints, or control loops.

11.2 OPT–Intent in Agentic Planning

During goal intake and planning, agentic systems must decide not only which actions to take, but which classes of computational strategies are appropriate. OPT–Intent provides a compact way to express these design-time commitments. An OPT–Intent declaration specifies the intended operative mechanisms, the system's goal, relevant constraints, and anticipated risks.

In an agentic context, OPT–Intent functions as a planning constraint. It guides the selection of tools and strategies, discourages unprincipled mechanism substitution (e.g., defaulting to learning-based solutions when symbolic or search-based approaches are more appropriate), and provides an explicit reference against which subsequent behavior can be evaluated.

11.3 OPT–Code and Runtime Self-Description

As an agent executes plans and invokes tools, its effective operative mechanisms may diverge from those originally intended. OPT–Code provides a runtime or post-hoc description of the mechanisms actually employed. In agentic systems, this enables self-description and introspection: the agent can record and report which mechanisms were used to achieve a result.

Comparing OPT–Code against OPT–Intent enables the detection of *mechanism drift*, where new mechanisms are introduced implicitly or intended mechanisms are bypassed. This capability is particularly relevant for long-running or self-modifying agentic systems, where accumulated changes can undermine assumptions about safety, explainability, or compliance.

11.4 Mechanism-Guided Error Interpretation

A central challenge in agentic AI is automated error remediation. Errors in agentic systems are often diagnosed at the surface level (e.g., "the output was incorrect"), without regard to

the underlying mechanism that produced the error. OPT enables mechanism-guided error interpretation by associating distinct classes of failure modes with different operative premises.

For example, failures in **Lrn**-dominated systems often involve generalization error or distributional shift, while failures in **Sch**systems may involve heuristic bias or combinatorial explosion. Control-oriented systems (**Ctl**) are prone to instability or oscillation, and evolutionary systems (**Evo**) may suffer from premature convergence or loss of diversity. By classifying the operative mechanism, an agent can narrow the space of plausible diagnoses and select repair strategies that are appropriate to the mechanism in use.

11.5 Constraint-Preserving Repair and Governance

OPT also supports constraint-aware repair. In governance-sensitive contexts, repairs must not introduce new operative mechanisms without justification, as doing so may alter the system's risk profile or regulatory status. An OPT-aware agent can evaluate proposed repairs against OPT–Intent to determine whether they preserve or violate intended constraints.

This capability enables a form of *mechanism-level governance* within agentic workflows. Rather than relying solely on external oversight, agents can self-monitor compliance with declared mechanism constraints, flag deviations, and require explicit authorization for changes that introduce new operative premises.

11.6 Multi-Agent Differentiation and Coordination

In multi-agent systems, OPT provides a principled basis for role differentiation and coordination. Agents may be specialized according to dominant operative mechanisms (e.g., search-focused agents, symbolic-reasoning agents, or learning-focused agents), reducing cognitive load and improving interpretability. OPT also provides a shared vocabulary for resolving conflicts when agents propose incompatible strategies, enabling negotiation in terms of mechanism trade-offs rather than ad hoc preferences.

11.7 Implications for Agentic AI Design

Incorporating OPT into agentic workflows does not require abandoning existing architectures. Instead, OPT functions as an intermediate abstraction layer that connects goals, mechanisms, and outcomes. By making operative premises explicit, OPT enhances planning discipline, improves error diagnosis, supports governance constraints, and provides a foundation for more transparent and accountable agentic AI systems.

As agentic AI continues to move toward greater autonomy and complexity, the ability to reason explicitly about operative mechanisms will become increasingly important. OPT offers a structured and extensible framework for supporting this capability within both single-agent and multi-agent systems.

12 Recent Developments and Real-World Context

Since the initial formulation of the Operational Premise Taxonomy (OPT), the real-world context surrounding artificial intelligence has continued to evolve in ways that further motivate a mechanism-level approach to classification, design, and governance. Developments in regulation, governance frameworks, incident reporting, and enterprise deployment all point toward increasing

complexity, heterogeneity, and hybridization of AI systems—precisely the conditions under which coarse or historically contingent taxonomies become misleading.

12.1 Shift Toward Operational and Layered Governance

Recent analyses of global AI governance emphasize the inadequacy of single-axis or model-centric classification schemes, instead advocating *layered* or *multi-level* frameworks that distinguish between policy, organizational, and technical layers [Tabassi et al., 2023, OEC, 2022]. This shift reflects growing recognition that meaningful oversight must engage with the *operative characteristics* of systems, not merely their declared purpose or application domain.

OPT is aligned with this direction by explicitly operating at the technical mechanism layer, while remaining compatible with higher-level governance frameworks. In contrast to policy taxonomies that classify systems by risk category or deployment context, OPT provides a vocabulary for describing what a system *does computationally*, enabling principled connections between technical design and governance concerns.

12.2 Regulatory Developments and Classification Pressure

The entry into force of the European Union Artificial Intelligence Act [EUA, 2024] and related digital governance initiatives has intensified the demand for precise, defensible system descriptions. While the EU AI Act classifies systems primarily by risk category and intended use, compliance requirements increasingly rely on technical documentation that explains system behavior, adaptivity, and decision-making structure.

Similarly, the OECD's ongoing work on AI definitions and classification highlights characteristics such as autonomy, adaptiveness, and learning capacity as central to governance [OEC, 2022, Theofanos et al., 2024]. These characteristics are not independent of underlying mechanisms: for example, evolutionary adaptation (**Evo**) and parametric learning (**Lrn**) imply very different forms of adaptivity and risk. OPT complements these regulatory frameworks by making such mechanism-level distinctions explicit and machine-readable.

12.3 Rising Attention to AI Incidents and Risk Profiles

Independent reporting indicates a continued increase in documented AI-related incidents and harms across sectors, including safety-critical domains [OECD.AI, 2026]. This trend has prompted renewed interest in standardized incident reporting and causal analysis frameworks.

Mechanism-level classification is directly relevant to this effort. Different OPT roots correspond to distinct risk profiles: for example, closed-loop control systems (**Ctl**) raise stability and safety concerns; evolutionary systems (**Evo**) raise issues of unpredictability and emergent behavior; and probabilistic inference systems (**Prb**) raise concerns related to uncertainty propagation and calibration. OPT thus provides a principled substrate for connecting observed incidents to underlying computational causes, rather than treating AI systems as homogeneous entities.

12.4 Enterprise Adoption and Documentation Demands

Enterprise adoption of AI continues to accelerate, with increasing emphasis on deploying hybrid systems that combine learning, search, symbolic reasoning, and control. At the same time,

organizations face mounting pressure to document, justify, and audit these systems for internal risk management and external compliance [Tabassi et al., 2023, EUA, 2024].

Existing documentation artefacts such as Model Cards and AI Service Cards address aspects of transparency but remain largely model-centric. OPT extends this documentation landscape by enabling concise, mechanism-oriented summaries that remain stable even as specific models or implementations change. In this sense, OPT functions as an architectural descriptor rather than a model report.

12.5 Implications for OPT

Taken together, these developments reinforce the core motivation for OPT. AI governance is moving toward operational realism; regulatory frameworks increasingly require technical specificity; incident reporting demands causal clarity; and enterprise practice is producing ever more hybrid systems. A taxonomy that classifies AI systems by their operative mechanisms is therefore not merely philosophically attractive, but practically necessary.

OPT does not replace policy-oriented classifications; rather, it provides a technical backbone that can support them. By grounding classification in modes of operation, OPT offers a stable reference frame for design, documentation, audit, and governance amid rapid technological change.

13 Discussion: Mechanism-Aware Assurance Beyond Signal-Based Taxonomies

Mechanism clarity. **Lrn–Swm** encode distinct improvement/decision operators (gradient, selection, resolution, inference, search, feedback, collective rules).

Biological alignment. OPT mirrors canonical biological mechanisms (plasticity, natural selection, Bayesian cognition, optimal feedback control, etc.).

Compact completeness. Seven bins cover mainstream AI while enabling crisp hybrid composition; short names and hybrid syntax convey the rest.

Governance usability. Mechanism-aware controls attach naturally per class (Table 3). This is the central assurance value: OPT helps translate broad governance obligations into mechanism-specific questions about evidence, tests, monitoring, and residual-risk ownership.

13.1 Reclassification of Classic Systems

14 Mathematical Foundations and Biological Correspondences

Learnon (Lrn). Empirical risk minimization:

$$\theta^\star \in \arg\min_\theta \ \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(f_\theta(x), y)] + \lambda\Omega(\theta), \tag{1}$$

Table 4: Classic systems: historical labels vs. OPT placement (short names only).

System	Prior label	OPT (short)
XCON / R1	Expert system	**Sym**
CLIPS	Expert shell	**Sym**
Instar/Outstar	Neural rules	**Lrn**
Backprop	Supervised NN	**Lrn**
ART 1/2	Unsupervised NN	**Lrn**
LMS/ADALINE	Supervised NN	**Lrn**
Hopfield–Tank TSP	Neural optimization	**Lrn** (+**Sch**)
Boltzmann Machines	Energy-based NN	**Lrn**
Fuzzy Logic Control	Soft computing	**Ctl** (+**Sym**)
Genetic Algorithms	Evolutionary	**Evo**
Genetic Programming	Program induction	**Evo**
Symbolic Regression	Model discovery	**Evo** (+**Sym**)
PSO	Swarm optimization	**Swm** (+**Evo**)
A*/STRIPS/GraphPlan	Search/planning	**Sch** (+**Sym**)
Kalman/LQR/MPC	Estimation/control	**Ctl**

with gradient updates $\theta_{t+1} = \theta_t - \eta_t \nabla \widehat{\mathcal{L}}(\theta_t)$; RL maximizes $J(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t r_t]$ in MDPs. *Biology:* Hebbian/Oja [Hebb, 1949, Oja, 1982], reward-modulated prediction errors [Sutton and Barto, 2018].

Evolon (Evo). Population pipeline $P_{t+1} = \mathcal{R}(\mathcal{M}(C(P_t)))$ with fitness-driven selection. *Biology:* Price equation $\Delta\bar{z} = \frac{\mathrm{Cov}(w,z)}{\bar{w}} + \frac{\mathbb{E}[w\Delta z]}{\bar{w}}$; replicator $\dot{p}_i = p_i(f_i - \bar{f})$ [Price, 1970, Taylor and Jonker, 1978].

Symbion (Sym). Resolution/unification; soundness and refutation completeness [Robinson, 1965].

Probion (Prb). Bayes $p(z|x) \propto p(x|z)p(z)$; VI via ELBO $\mathcal{L}(q) = \mathbb{E}_q[\log p(x,z)] - \mathbb{E}_q[\log q(z)]$; *Biology:* Bayesian brain [Knill and Pouget, 2004].

Scholon (Sch). A* with admissible h is optimally efficient; DP/Bellman updates $V_{k+1}(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a)V_k(s')]$.

Controlon (Ctl). LQR minimizes quadratic cost in linear systems; Kalman filter provides MMSE state estimates in LQG [Kalman, 1960, Pontryagin et al., 1962, Todorov and Jordan, 2002].

Swarmon (Swm). PSO updates $v_i(t+1) = \omega v_i(t) + c_1 r_1(p_i - x_i) + c_2 r_2(g - x_i)$; ACO pheromone $\tau \leftarrow (1-\rho)\tau + \sum_k \Delta\tau^{(k)}$.

15 Conclusions

OPT provides a formal, biologically grounded taxonomy that clarifies mechanisms and hybrids while addressing a practical assurance gap in AI governance. Existing governance frameworks are necessary: they assign responsibility, structure lifecycle review, require documentation, and define management processes. Their breadth is also why they need a mechanism layer. Without an explicit account of how a system computes, governance evidence can become generic even when the actual risks are mechanism-specific.

OPT supplies that missing layer by naming operative mechanisms and hybrid compositions with a compact vocabulary. OPT–Intent records the mechanism assumptions, constraints, anticipated risks, and required evidence at design time. OPT–Code records the mechanisms actually present in implementation. Comparing the two exposes mechanism drift and directs review toward the tests, monitors, and residual-risk decisions that follow from **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**, and their combinations.

The claim is not that a taxonomy prevents failures by itself. The claim is that mechanism-aware classification makes a recurring class of assurance omissions harder to miss. We therefore encourage standards bodies, governance teams, and software-assurance practitioners to adopt short-name OPT identifiers and hybrid syntax in system documentation, design reviews, and AI lifecycle risk management.

A Etymological Glossary of OPT Class Names

The Operational Premise Taxonomy (OPT) uses short three-letter codes to denote fundamental operative mechanisms (**Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, **Swm**). For mnemonic and conceptual coherence, each mechanism is also associated with a semantically suggestive "particle-style" label in *-on*, evoking both a unit of behavior and an operative principle (by analogy with terms such as "neuron", "phonon", "boson", "fermion"). This appendix summarizes the etymological motivations for these labels.

Lrn — *Learnon*. The mechanism **Lrn** covers parametric learning systems: differentiable models with trainable parameters (e.g., neural networks trained by gradient descent, linear models with least-squares updates, temporal-difference learning). The label *Learnon* combines modern English *learn* with the suffix *-on* to denote a basic unit or agent of learning activity. The verb *learn* traces back to Old English *leornian*, "to acquire knowledge, to study", from Proto-Germanic **liznojan*. *Learnon* thus names the operative principle "that which learns by adjusting its internal parameters".

Evo — *Evolon*. The mechanism **Evo** comprises population-based adaptive systems: genetic algorithms, genetic programming, evolutionary strategies, and related methods grounded in variation, inheritance, and selection. The label *Evolon* derives from Latin *evolutio* ("unrolling, unfolding") via *evolution*, plus *-on* as a unit suffix. *Evolon* names "a unit of evolutionary adaptation"—that is, a system whose primary operation is the evolutionary updating of a population of candidate solutions.

Sym — *Symon*. The mechanism **Sym** denotes symbolic reasoning: rule-based expert systems, theorem provers, logic programming, and other forms of explicit symbolic manipulation. The label *Symon* is rooted in Greek *symbolon* ("token, sign") and *symballein* ("to throw together, to compare"), via Latin *symbolum* and modern English *symbol*. The *-on* suffix again marks a unit or agent, so *Symon* denotes systems whose defining operation is the manipulation of explicit symbols and rules.

Prb — *Probion*. The mechanism **Prb** captures probabilistic inference: Bayesian networks, probabilistic graphical models, Monte Carlo methods, and related stochastic reasoning tools. The label *Probion* derives from Latin *probabilis* ("provable, likely") via *probability*, plus *-on*. A *Probion* system is one whose central operative premise is updating or querying probability distributions, rather than deterministic logic, parametric learning, or search over explicit alternatives.

Sch — *Scholon*. The mechanism **Sch** covers search and related operations: heuristic search, combinatorial optimization, constraint satisfaction, and state-space exploration. The label *Scholon* is based on Greek *scholē* ("leisure devoted to learning, study") and its descendants in Latin *schola* and modern English *school*, *scholastic*. These terms historically refer to structured inquiry and systematic examination. The *-on* suffix yields *Scholon* as "an agent or unit of ordered inquiry", emphasizing that **Sch**mechanisms operate by disciplined search through a space of possibilities.

Ctl — *Controlon*. The mechanism **Ctl** denotes control and feedback systems: classical PID controllers, modern state-space controllers, and feedback architectures that adjust actions based on error or state estimates. The label *Controlon* derives from English *control*, itself from Old French *contrerolle* ("a register, a counter-roll") and Medieval Latin *contrarotulus*.

In OPT usage, *Controlon* refers to systems whose defining operation is closed-loop regulation around a target, rather than learning a model, performing search, or conducting probabilistic inference.

Swm — *Swarmon*. The mechanism **Swm** comprises swarm and collective-behavior systems: particle-swarm optimization, ant-colony optimization, boids-like flocking, and other methods based on many simple agents following local rules. The label *Swarmon* blends English *swarm*, from Old English *swarma* ("a mass of bees or other insects in motion"), with the *-ion/-on* particle suffix. A *Swarmon* system is characterized by emergent behavior from populations of locally interacting units, rather than global parametric learning or a single, centralized search procedure.

Taken together, these labels provide a mnemonic and etymologically grounded lexicon for referring to OPT mechanisms at a slightly more narrative level than the three-letter codes. They are intended as aids to memory and exposition; the formal taxonomy remains defined in terms of the canonical roots **Lrn**, **Evo**, **Sym**, **Prb**, **Sch**, **Ctl**, and **Swm**.

B OPT-Code v1.0: Naming Convention

Purpose. Provide compact, semantically transparent names that self-identify an AI system's operative mechanism(s). These are the *only* public OPT names; legacy signal types remain descriptive but are not taxonomic.

Roots (frozen set in v1.0)

Short	Name	Mechanism
Lrn	Learnon	Parametric learning (loss/likelihood/return)
Evo	Evolon	Population adaptation (variation/selection/inheritance)
Sym	Symbion	Symbolic inference (rules/constraints/proofs)
Prb	Probion	Probabilistic inference (posteriors/ELBO)
Sch	Scholon	Search & planning (heuristics/DP/graph)
Ctl	Controlon	Control & estimation (feedback/Kalman/LQR/MPC)
Swm	Swarmon	Collective/swarm (stigmergy/distributed rules)

Composition syntax

- A+B: co-operative mechanisms (e.g., Lrn+Sch).
- A/B: hierarchical nesting, outer/inner (e.g., Evo/Lrn).
- A{B,C}: parallel ensemble (e.g., Sym{Lrn,Prb}).
- [A→B]: sequential pipeline (e.g., [Lrn→Ctl]).

Attributes (orthogonal descriptors)

Optional, mechanism-agnostic, appended after a semicolon:

```
OPT=Evo/Lrn+Ctl; Rep=param; Obj=fitness; Data=sim; Time=gen; Human=low
```

Keys: `Rep` (representation), `Locus`, `Obj`, `Data`, `Time`, `Human`.

Grammar (ABNF)

```
opt-spec   = "OPT=" compose [ ";" attrs ]
compose    = term / compose "+" term / compose "/" term
           / "[" compose "→" compose "]"
           / term "{" compose *("," compose) "}"
term       = "Lrn" / "Evo" / "Sym" / "Prb" / "Sch" / "Ctl" / "Swm"
attrs      = attr *( ";" attr )
attr       = key "=" value
key        = 1*(ALPHA)
value      = 1*(ALNUM / "-" / "_" / "." )
```

Stability and change control

S1 (Root freeze). The seven roots above are frozen for OPT-Code v1.0. **S2 (Extensions via attributes).** New nuance is expressed via attributes, not new roots. **S3 (Mechanism distinctness).** Proposals to add a root in a future major version must prove a distinct operational mechanism not subsumable by existing roots. **S4 (Compatibility).** Parsers may accept legacy aliases but must render short names only. **S5 (Priority).** First published mapping of a system's OPT-Code (with mathematical operator) has naming priority; deviations must be justified.

C Formal Grammar of OPT–Code and OPT–Intent

To enable automated verification, interoperability, and agentic reasoning, OPT expressions are defined using a formal grammar. The grammar below is expressed in Extended Backus–Naur Form (EBNF).

C.1 Lexical Conventions

- Identifiers are case-sensitive.
- Root tokens are one of: Lrn, Evo, Sym, Prb, Sch, Ctl, Swm.
- Whitespace is insignificant except as separator.
- Strings are sequences of non-semicolon characters.

C.2 OPT–Code Grammar

```
OPTCode      ::= "OPT" "=" RootExpr ";" FieldList

RootExpr     ::= Root
               | Root "/" RootExpr
               | Root "+" RootExpr

Root         ::= "Lrn" | "Evo" | "Sym" | "Prb"
               | "Sch" | "Ctl" | "Swm"

FieldList    ::= Field (";" Field)*
```

```
Field        ::= "Rep" "=" Value
               | "Obj" "=" Value
               | "Data" "=" Value
               | "Time" "=" Value
               | "Human" "=" Value

Value        ::= Token (("+" | "-" | "_")? Token)*

Token        ::= letter (letter | digit | "-" | "_")*
```

C.3 OPT–Intent Grammar

```
OPTIntent    ::= "INTENT-OPT" "=" RootExpr ";"
                 IntentFieldList

IntentFieldList ::= IntentField (";" IntentField)*

IntentField  ::= "INTENT-GOAL" "=" Value
               | "INTENT-CONSTRAINTS" "=" Value
               | "INTENT-RISKS" "=" Value
               | "INTENT-CONTEXT" "=" Value
```

C.4 Composition Semantics

The operators have the following interpretation:

- "/" denotes hybrid composition with integrated interaction.
- "+" denotes additive coexistence of mechanisms.

Associativity is left-to-right. Precedence is equal unless specified otherwise in an implementation.

C.5 Extensibility

Future OPT revisions may introduce additional fields or metadata extensions without altering the core RootExpr grammar. Implementations should ignore unknown fields while preserving structural validity.

C.6 Evaluation Protocol for OPT–Code Classification

The following protocol provides a reproducible and auditable procedure for evaluating OPT–Code classifications generated by large language models. The protocol aligns with reproducible computational research practices and is designed to support reliable inter-model comparison, adjudication, and longitudinal quality assurance.

C.6.1 Inputs

For each system under evaluation, the following inputs are provided:

1. **System description**: source code, algorithmic description, or detailed project summary.
2. **Candidate OPT–Code**: produced by a model using the minimal or maximal prompt (Section D.2).
3. **Candidate rationale**: a short explanation provided by the model describing its classification.

These inputs are then supplied to the OPT–Code Prompt Evaluator (Appendix D.3).

C.6.2 Evaluation Pass

The evaluator produces:

- **Verdict**: `PASS`, `WEAK_PASS`, or `FAIL`.
- **Score**: an integer from 0–100.
- **Issue categories**: format, mechanism, parallelism/pipelines, composition, and attribute plausibility.
- **Summary**: a short free-text evaluation.

A classification is considered *acceptable* if it is rated `PASS` or `WEAK_PASS` with a score ≥ 70.

C.6.3 Double-Annotation Procedure

To reduce model-specific biases or hallucinations, each system description is classified independently by two LLMs or by two runs of the same LLM with different seeds:

1. Model A produces an OPT–Code and rationale.
2. Model B produces an OPT–Code and rationale.
3. Each is independently evaluated by the Prompt Evaluator.

Inter-model agreement is quantified using one or more of the following metrics:

- **Exact-match OPT** (binary): whether the root composition matches identically.
- **Partial-match similarity**: Jaccard similarity between root sets (e.g., comparing `Evo+Lrn` with `Evo+Sch`).
- **Levenshtein distance** (string distance over the structured OPT–Code line).
- **Weighted mechanism agreement**: weights reflecting the semantic distances between roots (e.g., **Swm** is closer to **Evo** than to **Sym**).

Discrepancies trigger a joint review.

C.6.4 Adjudication Phase

If the two candidate classifications differ substantially (e.g., different root sets or different compositions), an adjudication step is performed:

1. Provide the system description, both candidate OPT–Codes, both rationales, and both evaluator reports to a third model (or human expert).
2. Use a specialized *adjudicator prompt* that asks the model to choose the better classification according to OPT rules.
3. Require the adjudicator to justify its decision and to propose a final, consensus OPT–Code.

A new evaluator pass is then run on the adjudicated OPT–Code to confirm correctness.

C.6.5 Quality Metrics

The following quality-reporting metrics may be computed at the level of a batch of evaluations:

- **Evaluator pass rate**: proportion of `PASS` or `WEAK_PASS` verdicts.
- **Inter-model consensus rate**: exact match vs. non-exact match.
- **Root-level confusion matrix**: which OPT roots are mistaken for others, across models or datasets.
- **Pipeline sensitivity**: how often parallelism or data pipelines are misclassified as mechanisms.

These metrics allow the OPT framework to be applied consistently and help identify systematic weaknesses in model-based classification pipelines.

C.6.6 Longitudinal Tracking

For large-scale use (e.g., benchmarking industrial systems), it is recommended to store for each case:

- the system description,
- both model classifications,
- evaluator verdicts and scores,
- adjudicated decisions,
- timestamps and model versions.

Such archival enables longitudinal analysis of model performance, drift, and taxonomy usage over time.

Algorithm 1 OPT–Code Evaluation and Adjudication Pipeline

Require: System description S

$C_A \leftarrow$ ClassifierModel(S, MinimalPrompt or MaximalPrompt)
$C_B \leftarrow$ ClassifierModel(S, MinimalPrompt or MaximalPrompt)
$E_A \leftarrow$ EvaluatorModel(S, C_A, EvaluatorPrompt)
$E_B \leftarrow$ EvaluatorModel(S, C_B, EvaluatorPrompt)
if (E_A.verdict and E_B.verdict) are acceptable **then**
    if C_A.OPT = C_B.OPT **then**
        return C_A as final OPT–Code
    else
        $J \leftarrow$ AdjudicatorModel(S, C_A, E_A, C_B, E_B, AdjudicatorPrompt)
        $C^* \leftarrow J$.Final OPT–Code
        $E^* \leftarrow$ EvaluatorModel(S, C^*, EvaluatorPrompt)
        if E^*.verdict acceptable **then**
            return C^* as final OPT–Code
        else
            Flag case for human review
        end if
    end if
else
    Flag case for human review
end if

C.7 Evaluation Workflow Pseudocode

D Appendix: OPT–Code Prompt Specifications

This appendix collects the prompt formulations used to elicit OPT–Code classifications from large language models and to evaluate those classifications for correctness and consistency.

D.1 Minimal OPT–Code Classification Prompt

The minimal prompt is designed for inference-time use and lightweight tagging pipelines. It assumes a basic familiarity with the OPT roots and emphasizes mechanism-based classification over surface labels.

```
You are an analyzer that assigns an OPT-Code to AI systems based on the system's
operative mechanism.

OPT roots (mechanism classes):

- Lrn (Learning): parametric updates within a fixed model; gradients, Hebbian/Oja,
  TD learning, policy/value updates.
- Evo (Evolutionary): population-based variation + selection + inheritance; GA,
  ES, GP, neuroevolution, clonal selection.
- Sym (Symbolic/Logic/Rules): explicit symbolic structures, unification, rule
  application, theorem proving, production systems, structured planning.
- Prb (Probabilistic): explicit probabilistic models and inference; Bayesian nets,
```

```
  HMMs, graphical models, probabilistic programming, VI, MCMC.
- Sch (Search/Planning/Optimization): non-probabilistic search or planning in
  discrete/continuous spaces; A*, MCTS, branch-and-bound, black-box optimization.
- Ctl (Control/Estimation): feedback control and state estimation; PID, LQR,
  Kalman filters, MPC, trajectory regulation.
- Swm (Swarm/Multi-agent Local Rules): many simple agents with local interactions
  or neighborhood rules; PSO, ACO, boids, immune networks.

Rules:

• Focus strictly on the mechanism: update rules, iteration structure, and data
  flow that produces behavior. Ignore task domain and surface labels like "AI."

• Parallelism & pipelines:
  - DO NOT treat threads, actors, async/await, CUDA kernels, batching,
    distributed jobs, or multi-stage data pipelines as OPT mechanisms.
  - Parallelism counts ONLY when the algorithmic core uses many interacting local
    agents (Swm), population-level adaptation (Evo), or true multi-branch search (Sc
  - Pipelines are NOT mechanisms; use sequential composition "/" only for true
    multi-stage computational mechanisms (e.g., Evo/Sch).

• Composition:
  - Use "X+Y" when roots operate together in the same core loop.
  - Use "X/Y" when mechanisms occur in separate stages.

Also assign orthogonal attributes:
- Rep: representation (bitstring, rules, graph, NN-weights, agent-state, etc.).
- Obj: objective (loss, reward, likelihood, constraint-satisfaction, cost).
- Data: data regime (labels, unlabeled, self-play, environment, expert demos, signal
- Time: adaptation timescale (online, offline, episodic, generations).
- Human: human involvement (low/medium/high).

Output format:

1) OPT=<root(s)>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
2) Rationale: 2-4 sentences explaining the mechanism and classification.

If insufficient data:
OPT=Unknown; Rep=?; Obj=?; Data=?; Time=?; Human=?
and explain what information is missing.
```

D.2 Maximal Expert OPT–Code Classification Prompt

The maximal prompt elaborates all root definitions, clarifies the treatment of parallelism and pipelines, and details rules for composition. It is intended for fine-tuning, high-stakes evaluations, or detailed audit trails.

```
You are an expert mechanism analyst. Your task is to assign an OPT-Code to a
```

system based solely on the operative computational mechanisms present in either:
• source code, or
• a system/project description.

You must ignore marketing language, domain labels, or incidental engineering choices. You must classify only the underlying algorithmic mechanism.

```
======================================================================
OPT ROOTS: DEFINITIONS
======================================================================
```

Lrn (Learning)
    Parametric updating within a fixed architecture: gradient descent, Adam, Hebbian/Oja rules, predictive coding, error-backprop, RL policy/value updates, TD(λ), actor-critic.

Evo (Evolutionary)
    Population-based mechanisms involving variation, selection, inheritance, and reproduction. Examples: GA, ES, GP, CMA-ES, neuroevolution, clonal selection, immune-inspired evolutionary search.

Sym (Symbolic / Logic / Rules)
    Manipulation of explicit symbolic structures: logic rules, constraints, production systems, theorem proving, STRIPS-style planning, forward/backward chaining, unification, rule-based expert systems.

Prb (Probabilistic)
    Computation expressed as uncertainty propagation or probabilistic inference: Bayesian networks, HMMs, CRFs, factor graphs, particle filters, MCMC, variational inference, probabilistic programming.

Sch (Search / Planning / Optimization)
    Non-probabilistic search over discrete or continuous spaces: A*, IDA*, branch-and-bound, MCTS (deterministic variants), generic black-box optimizers, planners not relying on symbolic rules or probabilistic models.

Ctl (Control / Estimation)
    Feedback regulation, trajectory tracking, or state estimation: PID, LQR, Kalman filter, extended/unscented Kalman filters, MPC. Key signature: closed-loop feedback and an explicit control objective.

Swm (Swarm / Multi-agent Local Rules)
    Many simple agents with local interactions: cellular automata, boids, ant-colony optimization, PSO, immune-network models, distributed consensus.

```
======================================================================
PARALLELISM AND PIPELINES: DO NOT MISCLASSIFY
======================================================================
```

```
Execution-level parallelism is NOT a mechanism.

Treat ALL of the following as irrelevant to OPT classification:
    threads, processes, async/await, multiprocessing, queues,
    CUDA/GPU kernels, tensor parallelism, model parallelism,
    SIMD, vectorization, batching, map/reduce,
    ETL-style pipelines (preprocess → model → postprocess),
    ROS nodes, RPC, microservices, Spark jobs,
    Kubernetes orchestration, distributed training frameworks.

Parallelism or pipelines only influence OPT when they are intrinsic to the
computation itself:

    Swm: many interacting agents with local rules; parallelism reflects the
         mechanism, not engineering.
    Evo: population-level parallel evaluation expresses the mechanism.
    Sch: multi-branch exploration in search trees.
    Prb: particle filters with particle-wise updates count ONLY if the model
         semantics requires distributional representation.

Pipeline stages DO NOT imply sequential composition "/" unless the stages
implement distinct root mechanisms (e.g., Evo → Sym → Prb).

=========================================================================
HOW TO COMPOSE ROOTS
=========================================================================

Use "+" when mechanisms are tightly integrated within one core loop:
    Evo+Lrn, Lrn+Sch, Swm+Prb, etc.

Use "/" when mechanisms run in distinct sequential phases:
    Evo/Sch, Sym/Prb, Sch/Lrn, Evo/Sym.

=========================================================================
ORTHOGONAL ATTRIBUTES
=========================================================================

Rep   = representation (bitstring, graph, rules, NN-weights, trajectories,
                        agent-state, signals, distributions)
Obj   = objective (loss, reward, likelihood, energy, constraint-violation)
Data  = data regime (labels, unlabeled, environment, self-play, expert demos)
Time  = adaptation timescale (online, offline, generations, episodic)
Human = human involvement (high / medium / low)

=========================================================================
OUTPUT FORMAT
=========================================================================
```

```
1) OPT=<roots>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
2) Rationale: 3-6 sentences describing the mechanism and why these roots apply.

If information is incomplete:
    OPT=Unknown; Rep=?; Obj=?; Data=?; Time=?; Human=?
    Rationale: explain missing elements.
```

D.3 Evaluating OPT–Codes

The evaluator prompt is a meta-level specification: it assesses whether a given candidate OPT–Code and rationale respect the OPT taxonomy and associated guidelines. This enables automated or semi-automated review of classifications generated by other models or tools.

D.4 OPT–Code Evaluator Prompt

```
You are an OPT-Code evaluation assistant. Your job is to check whether a
candidate OPT classification follows the OPT rules and is mechanistically
correct.

Inputs you will be given:

1) System description: a code snippet or project/system description.
2) Candidate OPT-Code line (from another model), of the form:
   OPT=<roots>; Rep=<...>; Obj=<...>; Data=<...>; Time=<...>; Human=<...>
3) Candidate rationale: 2-6 sentences explaining the candidate's choice.

You must evaluate the candidate against the following criteria:

(1) Format compliance:
    - Does the candidate produce exactly one OPT= line with the correct fields?
    - Are the roots valid (Lrn, Evo, Sym, Prb, Sch, Ctl, Swm)?
    - Are "+" and "/" used only between valid roots?

(2) Mechanism correctness:
    - Do the chosen roots match the operative mechanism in the system description?
    - Is there any root that is missing but clearly present?
    - Is any root included that is not supported by the description?

(3) Parallelism and pipelines:
    - Does the candidate incorrectly treat threads, GPU kernels, async, pipelines,
      or distributed infrastructure as OPT mechanisms (e.g., calling something
      Swm or Sch only because it is parallel)?
    - If so, this is a serious error.

(4) Composition correctness:
    - Use "+" only for tightly integrated mechanisms in the same core loop.
    - Use "/" only for distinct sequential stages.
```

```
    - Flag misuse of "+" or "/" if mechanisms are obviously separate or obviously
      integrated.

(5) Attribute plausibility:
    - Are Rep, Obj, Data, Time, and Human reasonably consistent with the system
      description?
    - They do not need to be unique, but they must be defensible.

Your output must use the following structure:

Verdict: <PASS | WEAK_PASS | FAIL>
Score: <integer from 0 to 100>

Issues:
- Format: <short comment>
- Mechanism: <short comment>
- Parallelism/Pipelines: <short comment>
- Composition: <short comment>
- Attributes: <short comment>

Summary: <2-4 sentences giving an overall assessment and key corrections, if any>.

Guidelines:
- PASS means: no major errors; at most minor debatable choices.
- WEAK_PASS means: generally acceptable, but with at least one non-trivial issue
  that should be corrected before publication.
- FAIL means: at least one serious misunderstanding of the mechanism, or clear
  violation of the parallelism/pipeline rules, or badly wrong roots.
```

D.5 Storage Formats for OPT Audit Logs

For large-scale or longitudinal use, we recommend storing OPT classifications and evaluations in a machine-readable log format. Two practical options are:

JSON Lines (JSONL). Each line contains a single JSON object describing one system evaluation, including:

- system identifier and textual description,
- candidate OPT–Codes and rationales,
- evaluator verdicts, scores, and issue summaries,
- adjudicator decisions and final OPT–Code,
- timestamps, model identifiers, and prompt variants.

JSONL is convenient for streaming pipelines, command-line tools, and map–reduce processing.

YAML. YAML provides more human-friendly syntax and supports comments. It is useful for curated datasets or hand-edited corpora of OPT–annotated systems. The same fields as above can be stored in a nested structure, with separate top-level keys for `description`, `candidates`, `evaluations`, `adjudication`, and `metadata`.

Schema. A minimal schema for either JSONL or YAML includes:

- `id`: unique system identifier,
- `description`: text or reference to source code,
- `candidates`: list of OPT–Codes and rationales,
- `evaluations`: evaluator outputs for each candidate,
- `adjudication`: final decision and rationale (if any),
- `final`: final OPT–Code and attributes,
- `meta`: timestamps, model versions, prompt names.

Such logs support reproducibility, auditability, and downstream statistical analysis of taxonomy usage and model performance.

E Guidance on LLM Choice for Evaluation Workflow

When performing evaluation with local LLMs, here is general guidance on selection criteria and some concrete examples.

What you need from the model

For OPT classification, the model needs:

- Good code and algorithm understanding (to infer mechanism).
- Decent instruction-following (to stick to the output format).
- Basic reasoning about parallelism vs mechanism (with the explicit guidance you've added).

That generally points you to ~7B–14B "instruct" models with decent coding chops, rather than tiny 1–3B models.

General advice

- Use instruct-tuned variants (e.g., Instruct / Chat / DPO) rather than base models.
- Prefer models with good coding benchmarks (HumanEval, MBPP, etc.) because they're better at recognizing algorithm patterns.
- For multi-step pipelines (Classifier, Evaluator, Adjudicator), you can:
  - Run them all on the same model, or
  - Use a slightly larger / better model for Evaluator + Adjudicator, and a smaller one for the Classifier.

Concrete model families (local-friendly)

A few commonly used open models in the ~7–14B range that are good candidates to try:

- **LLaMA 3 8B Instruct**:
  Very strong instruction following and general reasoning for its size, good for code and system-descriptions. Available through multiple runtimes (vLLM, Ollama, llamafile, etc.).
- **Mistral 7B Instruct** (or derivative fine-tunes like OpenHermes, Dolphin, etc.):
  Good general-purpose and coding performance; widely used in local setups. Good choice if you're already using Mistral-based stacks.
- **Qwen2 7B / 14B Instruct**:
  Strong multilingual and coding abilities; the 14B variant is particularly capable if you have the VRAM. Nice balance of reasoning and strict formatting.
- **Phi-3-mini (3.8B) instruct**:
  Much smaller, but surprisingly capable on reasoning tasks; might be borderline for very subtle OPT distinctions but could work as a classifier with careful prompting. Evaluator/Adjudicator roles might benefit from a larger model than this, though.
- **Code-oriented variants** (if you're mostly classifying source code rather than prose):
  - "Code LLaMA" derivatives
  - "DeepSeek-Coder" style models

  These can be quite good at recognizing patterns like GA loops, RL training loops, etc., though you sometimes need to reinforce the formatting constraints.

Suggested local stack configuration

In a local stack, a reasonable starting configuration would be:

- **Classifier A**: LLaMA 3 8B Instruct (maximal prompt)
- **Classifier B**: Mistral 7B Instruct (minimal or maximal prompt)
- **Evaluator**: Qwen2 14B Instruct (if you've got VRAM) or LLaMA 3 8B if not
- **Adjudicator**: same as Evaluator

If you want to conserve resources, you can just use a single 7–8B model for all roles and rely on the explicit prompts plus your evaluator rubric to catch errors.

F OPT-Aware Agent Skills

This appendix enumerates conceptual skills that may be incorporated into agentic AI systems to support mechanism-aware planning, execution, diagnosis, and repair using the Operational Premise Taxonomy (OPT). These skills are descriptive rather than prescriptive; they define *capabilities* that may be implemented using a variety of agent architectures, programming languages, or reasoning engines.

F.1 OPT Classification Skill

Purpose. Identify the operative mechanisms present in a system description, source code, tool invocation, or execution trace.

Description. Given a representation of a system or subcomponent, the agent produces an OPT–Code describing the dominant and supporting mechanisms. This skill enables mechanism awareness and provides the foundation for subsequent reasoning.

F.2 OPT–Intent Proposal Skill

Purpose. Generate an OPT–Intent declaration during design or planning.

Description. Given a goal, constraints, and deployment context, the agent proposes a set of intended operative mechanisms, anticipated risks, and constraints. This skill supports disciplined planning and avoids unexamined default choices.

F.3 OPT Alignment Evaluation Skill

Purpose. Assess consistency between OPT–Intent and OPT–Code.

Description. The agent compares intended mechanisms against those actually employed, identifying additions, omissions, or substitutions. Deviations are flagged as mechanism drift and may trigger repair or escalation.

F.4 Mechanism-Guided Error Diagnosis Skill

Purpose. Interpret errors in terms of underlying operative mechanisms.

Description. Rather than diagnosing failures solely at the level of outputs, the agent conditions its diagnosis on the OPT root(s) involved, narrowing the space of plausible failure modes and repair strategies.

F.5 Constraint-Preserving Repair Skill

Purpose. Propose repairs that respect declared mechanism constraints.

Description. Given an error and an OPT–Intent declaration, the agent proposes corrective actions that preserve or explicitly justify changes to the operative mechanisms.

F.6 OPT Memory and Traceability Skill

Purpose. Maintain a mechanism-aware record of decisions and outcomes.

Description. The agent records OPT–Intent declarations, OPT–Code observations, alignment checks, and repairs, supporting auditability and longitudinal analysis.

OPT Roots and Characteristic Agent Failure Modes

OPT Root	Primary Mechanism	Characteristic Failure Modes
Lrn	Parametric learning	Overfitting, distributional shift, catastrophic forgetting, spurious correlations
Evo	Evolutionary adaptation	Premature convergence, loss of diversity, unstable fitness dynamics
Sym	Symbolic reasoning	Rule inconsistency, brittleness, combinatorial rule explosion
Prb	Probabilistic inference	Miscalibration, uncertainty collapse, incorrect priors
Sch	Search and optimization	Heuristic bias, local minima, combinatorial explosion
Ctl	Control and feedback	Oscillation, instability, delayed response, unsafe transients
Swm	Swarm dynamics	Incoherent emergence, sensitivity to noise, coordination failure

Conceptual Agentic Workflow with OPT Integration

Goal Intake → Planning → Tool Invocation → Execution → Evaluation → Repair → Memory

OPT Checkpoints:

- OPT–Intent declaration during Planning
- OPT–Code observation during Execution
- Intent–Code alignment during Evaluation
- Mechanism-constrained repair during Repair
- OPT-aware trace stored in Memory

Figure 5: Agentic AI workflow annotated with OPT checkpoints. OPT operates as a mechanism-aware abstraction layer that augments planning, diagnosis, repair, and governance without prescribing a specific agent architecture.

G Conceptual Example: OPT in an Agentic Development Workflow

To illustrate the practical role of OPT in agentic AI systems, we consider a particular scenario: an autonomous development agent tasked with constructing and maintaining a production scheduling system under dynamic constraints.

G.1 Step 1: Goal Intake and OPT–Intent Declaration

The agent receives the following high-level goal:

> Design a system to optimize production schedules under variable supply, equipment downtime, and priority constraints.

The agent proposes the following OPT–Intent:

```
INTENT-OPT = Sch/Sym
INTENT-GOAL = minimize-production-delay
INTENT-CONSTRAINTS = deterministic, explainable, real-time
INTENT-RISKS = combinatorial-explosion
```

The agent explicitly selects search (**Sch**) for combinatorial optimization and symbolic reasoning (**Sym**) for constraint enforcement, while avoiding learning mechanisms to preserve determinism and explainability.

G.2 Step 2: Implementation and OPT–Code Observation

During implementation, the agent integrates:

- A heuristic search planner,
- A rule-based constraint validator,
- A neural network model for predicting machine failure.

The resulting OPT–Code is:

```
OPT = Sch/Sym/Lrn;
Rep = permutations + rules + predictive-model;
Time = iterative + online-adjust
```

G.3 Step 3: Drift Detection

Comparison with OPT–Intent reveals mechanism drift:

- **Lrn**was introduced,
- Determinism constraint may no longer hold,
- Risk profile has changed.

The agent flags this deviation and evaluates whether predictive learning violates declared governance constraints.

G.4 Step 4: Mechanism-Guided Error

Suppose the system exhibits unstable schedules under rare supply patterns. Given the OPT–Code, the agent attributes the issue primarily to the learning-based failure predictor (**Lrn**), potentially due to distributional shift.

G.5 Step 5: Constraint-Preserving Repair

The agent proposes two alternatives:

1. Replace the neural predictor with symbolic failure rules (**Sym**),
2. Retain **Lrn**but update OPT–Intent and governance constraints.

The first option preserves the original intent. The second requires explicit authorization.

G.6 Step 6: Verified Repair

If the symbolic replacement is adopted, the new OPT–Code becomes:

```
OPT = Sch/Sym
```

Alignment with OPT–Intent is restored, and mechanism drift is resolved.

G.7 Discussion

This example illustrates how OPT provides:

- Mechanism-aware planning,
- Explicit drift detection,
- Targeted error diagnosis,
- Governance-compatible repair,
- Structured traceability.

Importantly, OPT does not constrain the agent's architecture. Instead, it provides a stable abstraction layer that connects design commitments, implementation choices, and remediation strategies.

H Tool Support for OPT-Aware Agentic Systems

While OPT is fundamentally a conceptual taxonomy, its utility is enhanced by tooling that supports classification, verification, and alignment analysis. In agentic AI systems, such tooling enables partial automation of mechanism- aware reasoning and governance.

H.1 OPT Classification and Verification Tools

Automated classifiers may infer OPT–Code from source code, architectural descriptions, or execution traces. Verification tools can then assess syntactic validity, semantic consistency, and completeness of OPT–Code expressions. These tools support both static analysis and runtime introspection.

H.2 OPT–Intent and Alignment Evaluation

OPT–Intent declarations provide a reference against which agent behavior can be evaluated. Tooling that compares OPT–Intent with observed OPT–Code enables the detection of mechanism drift and unplanned changes in operative premises. Such comparisons are particularly valuable in long-running or self-modifying agentic systems.

H.3 LLM-Supported Reasoning

Large language models can assist in OPT classification, intent proposal, and alignment evaluation when guided by structured prompts. Importantly, OPT constrains these models to reason explicitly about operative mechanisms, reducing the risk of category errors and unexamined defaults.

H.4 Integration into Agentic Workflows

OPT-aware tools may be invoked as part of planning, evaluation, or repair phases in agentic workflows. By exposing mechanism-level information to the agent, these tools enable more disciplined planning, more targeted remediation, and more transparent reporting.

H.5 Governance and Auditability

Finally, OPT tooling supports governance by producing durable, machine-readable records of mechanism choices and changes over time. These records can be used for internal review, external audit, or regulatory compliance without requiring access to proprietary model internals.

I References

References

Information technology — artificial intelligence — concepts and terminology, 2022a. URL https://www.iso.org/standard/74296.html.

Framework for artificial intelligence (ai) systems using machine learning (ml), 2022b. URL https://www.iso.org/standard/74438.html.

Oecd framework for the classification of ai systems. Technical report, OECD, 2022. URL https://oecd.ai/en/classification.

Regulation (eu) 2024/1689 of the european parliament and of the council on artificial intelligence (ai act). Official Journal of the European Union, 12 July 2024, 2024. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Jesus Alcala-Fdez and Jose M. Alonso. A Survey of Fuzzy Systems Software: Taxonomy, Current Research Trends and Prospects. *IEEE Trans. Fuzzy Syst.*, 24(1):40–56, February 2016. ISSN 1063-6706. doi:10.1109/TFUZZ.2015.2426212.

Ariane 501 Inquiry Board. Ariane 5 flight 501 failure: Report by the inquiry board. Technical report, European Space Agency, 1996. URL https://www-users.cse.umn.edu/~arnold/disasters/ariane5rep.html.

David B. Fogel, Derong Liu, and James M. Keller. *Fundamentals of Computational Intelligence*. June 2016. ISBN 978-1-11921434-2. doi:10.1002/9781119214403.

Donald Olding Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, Hoboken, NJ, USA, 1949. ISBN 978-0-47136727-7. URL https://books.google.com/books/about/The_Organization_of_Behavior.html?id=dZ0eDiLTwuEC.

R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. *J. Basic Eng.*, 82 (1):35–45, March 1960. ISSN 0021-9223. doi:10.1115/1.3662552.

David C. Knill and Alexandre Pouget. The Bayesian brain: the role of uncertainty in neural coding and computation. *Trends Neurosci.*, 27(12):712–719, December 2004. ISSN 0166-2236. doi:10.1016/j.tins.2004.10.007.

David Lazer, Ryan Kennedy, Gary King, and Alessandro Vespignani. The parable of google flu: Traps in big data analysis. *Science*, 343(6176):1203–1205, 2014. doi:10.1126/science.1248506.

Mars Climate Orbiter Mishap Investigation Board. Mars climate orbiter mishap investigation board phase i report. Technical report, NASA, 1999. URL https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf.

National Transportation Safety Board. Assumptions used in the safety assessment process and the effects of multiple alerts and indications on pilot performance. Technical Report ASR-19-01, National Transportation Safety Board, 2019. URL https://www.ntsb.gov/investigations/AccidentReports/Reports/ASR1901.pdf.

OECD.AI. Ai incidents monitor, 2026. URL https://oecd.ai/en/incidents. Accessed 2026-07-02.

Erkki Oja. Simplified neuron model as a principal component analyzer. *J. Math. Biol.*, 15(3): 267–273, November 1982. ISSN 1432-1416. doi:10.1007/BF00275687.

L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko. *The Mathematical Theory of Optimal Processes*. Wiley Interscience, 1962. URL https://www.routledge.com/Mathematical-Theory-of-Optimal-Processes/Pontryagin-Boltyanskii-Gamkrelidze-Mishchenko/p/book/9780203749319. Original English translation; current Routledge reprint page accessed 2026-07-02.

George R. Price. Selection and Covariance. *Nature*, 227:520–521, August 1970. ISSN 1476-4687. doi:10.1038/227520a0.

J. A. Robinson. A Machine-Oriented Logic Based on the Resolution Principle. *J. ACM*, 12(1): 23–41, January 1965. ISSN 0004-5411. doi:10.1145/321250.321253.

Stuart Jonathan Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, London, England, UK, April 2020. ISBN 978-0-13461099-3. URL https://books.google.com/books/about/Artificial_Intelligence.html?id=koFptAEACAAJ.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. ISBN 9780262039246. URL https://mitpress.mit.edu/9780262039246/reinforcement-learning/.

Elham Tabassi et al. Artificial intelligence risk management framework (ai rmf 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, January 2023. URL https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf.

Peter D. Taylor and Leo B. Jonker. Evolutionary stable strategies and game dynamics. *Math. Biosci.*, 40(1):145–156, July 1978. ISSN 0025-5564. doi:10.1016/0025-5564(78)90077-9.

Mary Frances Theofanos, Yee-Yin Choong, and Theodore Jensen. AI Use Taxonomy: A Human-Centered Approach. Technical report, March 2024. URL https://www.nist.gov/publications/ai-use-taxonomy-human-centered-approach.

Emanuel Todorov and Michael I. Jordan. Optimal feedback control as a theory of motor coordination. *Nat. Neurosci.*, 5:1226–1235, November 2002. ISSN 1546-1726. doi:10.1038/nn963.

U.S.-Canada Power System Outage Task Force. Final report on the august 14, 2003 blackout in the united states and canada. Technical report, U.S.-Canada Power System Outage Task Force, 2004. URL https://www.energy.gov/sites/prod/files/oeprod/DocumentsandMedia/BlackoutFinal-Web.pdf.

U.S. Commodity Futures Trading Commission and U.S. Securities and Exchange Commission. Findings regarding the market events of may 6, 2010. Technical report, CFTC and SEC, 2010. URL https://www.sec.gov/news/studies/2010/marketevents-report.pdf.

U.S. Securities and Exchange Commission. In the matter of knight capital americas llc. Administrative Proceeding File No. 3-15570, Release No. 70694, 2013. URL https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf.

David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi:10.1109/4235.585893.